

Laboratorium 13: Hugging Face Ecosystem

Zadanie 1: Fine-tuning modelu BERT

Cel: Trening klasyfikatora binarnego zdań w oparciu o model BERT.

1. Studenci instalują bibliotekę transformers

```
pip install transformers
```

2. Studenci importują zbiór danych zawierający zdania angielskie z przypisanymi etykierami '1' – ciągi słów poprawne gramatycznie lub '0' – ciągi słów niepoprawne gramatycznie:

```
curl -L https://raw.githubusercontent.com/Denis2054/Transformers-for-NLP-2nd-Edition/master/Chapter03/in_domain_train.tsv --output "in_domain_train.tsv"
```

3. Tokenizacja, padding i wyznaczenie masek

a) każdą sekwencję ze zbioru danych należy uzupełnić tokenami początku i końca sekwencji np.:

```
"Neque porro quisquam est" -> "[CLS] Neque porro quisquam est [SEP]"
```

b) tokenizacja i kodowanie tokenów ze zbioru danych sentences:

```
from transformers import BertTokenizer
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased', do_lower_case=True)

tokenized_texts = [tokenizer.tokenize(sent) for sent in sentences]
input_ids = [tokenizer.convert_tokens_to_ids(x) for x in tokenized_texts]
```

c) padding i utworzenie masek – model BERT oczekuje na wejściu sekwencji tokenów o ustalonej długości; padding - należy znaleźć najdłuższy element zbioru input_ids , a następnie uzupełnić wszystkie pozostałe elementy tego zbioru zerami, tak, aby ich długość stała się równa długości maksymalnej, efektem operacji jest zbiór padded_ids; utworzenie masek attention_masks – dla każdego elementu zbioru padded_ids należy utworzyć listę attention_mask zawierającą 1 na pozycjach skojarzonych z niezerowymi tokenami i 0 na pozycjach w których zastosowano padding

4. Train test split, utworzenie datasetów i dataloaderów:

```
train_ids, test_ids, train_masks, test_masks, train_labels, test_labels = train_test_split(padded_ids,
                                                                                          attention_masks, labels, random_state=42, test_size=0.2, stratify = labels)

train_ids, val_ids, train_masks, val_masks, train_labels, val_labels = train_test_split(train_ids,
                                                                                          train_masks, train_labels, random_state=42, test_size=0.2, stratify = train_labels)

train_ids = torch.tensor(train_ids)
val_ids = torch.tensor(val_ids)
test_ids = torch.tensor(test_ids)

train_labels = torch.tensor(train_labels)
val_labels = torch.tensor(val_labels)
test_labels = torch.tensor(test_labels)
```

```

train_masks = torch.tensor(train_masks)
val_masks = torch.tensor(val_masks)
test_masks = torch.tensor(test_masks)

batch_size = 32

train_data = TensorDataset(train_ids, train_masks, train_labels)
train_dataloader = DataLoader(train_data, shuffle=True, batch_size=batch_size)

validation_data = TensorDataset(val_ids, val_masks, val_labels)
validation_dataloader = DataLoader(validation_data, shuffle=False, batch_size=batch_size)

test_data = TensorDataset(test_ids, test_masks, test_labels)
test_dataloader = DataLoader(test_data, shuffle=False, batch_size=batch_size)

```

5. Utworzenie instancji modelu BERT dla zadania klasyfikacji binarnej:

```

from transformers import BertForSequenceClassification

model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=2)

```

Przy tworzeniu instancji modelu generowane jest ostrzeżenie:

„Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight']

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.”

Ostrzeżenie dotyczy ostatniej warstwy modelu, która jest zainicjalizowana wartościami losowymi:

```

)
    (pooler): BertPooler(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (activation): Tanh()
    )
  )
  (dropout): Dropout(p=0.1, inplace=False)
  (classifier): Linear(in_features=768, out_features=2, bias=True)
)

```

6. Fine-tuning – wskazówki:

```

# optymalizator
optimizer = torch.optim.AdamW(model.parameters(),
                                lr = 2e-5,
                                eps = 1e-8
)

# Pętla treningowa
for step, batch in enumerate(train_dataloader):
    b_input_ids, b_input_mask, b_labels = batch
    optimizer.zero_grad()
    outputs = model(b_input_ids, token_type_ids=None,
                    attention_mask=b_input_mask, labels=b_labels)
    loss = outputs['loss']
    loss.backward()

```

```
optimizer.step()
```

7. Fine-tuning – należy wykonać trening w dwóch scenariuszach

- a) z zamrożonymi wszystkimi warstwami modelu BERT z wyjątkiem warstwy classifier
- b) z odmrożonymi wszystkimi warstwami

Należy porównać dokładności modeli dotrenowanych w obu scenariuszach.

Zadanie: Wykonaj trening modelu BERT dla zadania klasyfikacji binarnej.

Zadanie 2: BERT jako enkoder w zadaniu klasyfikacji binarnej

Cel: Demonstracja wykorzystania gęstej numerycznej reprezentacji tekstów w zadaniu klasyfikacji

1. Studenci implementują kod do enkodowania tekstów do postaci numerycznej:

```
from transformers import BertModel, BertConfig

# Initializing a model from the bert-base-uncased style configuration
# and accessing the model configuration
configuration = BertConfig()
model = BertModel(configuration)
configuration = model.config
print(configuration)

modelPreTrained = BertModel.from_pretrained("bert-base-uncased")
modelPreTrained = modelPreTrained.to(device)
modelPreTrained.eval()

for batch in validation_dataloader:
    b_input_ids, b_input_mask, b_labels = batch
    with torch.no_grad():
        features = modelPreTrained(b_input_ids, token_type_ids=None,
                                   attention_mask=b_input_mask)
```

Słownik features zawiera dwa klucze: 'last_hidden_state' i 'pooler_output'.

features['last_hidden_state'] jest tensorem o rozmiarze (BATCH_SIZE, MAX_LENGTH, 768) gdzie MAX_LENGTH jest maksymalną długością sekwencji tokenów w zbiorze danych (ustalona na etapie paddingu – patrz Zadanie 1). Element features['last_hidden_state'][i,j] jest reprezentacją numeryczną tokenu j w i-tym elemencie batcha – jest to kontekstowa reprezentacja tokena.

features['pooler_output'] jest tensorem o rozmiarze (BATCH_SIZE, 768). Element features['pooler_output'][i] jest numeryczną reprezentacją i-tego elementu batcha i zwykle jest to reprezentacja tokenu [CLS] danej sekwencji. Reprezentacji tej można używać jako wektora cech np. przy trenowaniu modeli „down-stream” NLP (np. klasyfikacja, NER, extractive QA)

2. Zbiór danych z Zadania 1 należy podzielić na część treningo-walidacyjną i testową, a następnie wykorzystać kod z pkt. 1 do przygotowania numerycznych reprezentacji tekstów – efektem realizacji punktu 2 jest zbiór danych, w którym każdemu zdaniu odpowiada wektor cech o długości 768 i odpowiednia etykieta 0/1

3. Implementacja, optymalizacja (w schemacie walidacji krzyżowej pięciokrotnej), trening i testy klasyfikatora MLP w oparciu o zbiór danych z pkt. 2

Zadanie: Wykonaj trening MLP w oparciu o reprezentację tekstów wyznaczoną z wykorzystaniem pretrenowanego modelu BERT. Porównaj wyniki z wynikami klasyfikacji w Zadaniu 1.

Zadanie 3: Enkodery w zadaniu wyszukiwania semantycznego

Cel: Demonstracja wykorzystania gęstej numerycznej reprezentacji tekstów w zadaniu wyszukiwania semantycznego

1. Studenci instalują bibliotekę sentence-transformers (<https://sbert.net/>)

```
pip install -U sentence-transformers
```

Biblioteka sentence transformers dostarcza wrappery na modele językowe, w szczególności na modele do generowania embeddingów. Sposób użycia modeli z sentence-transformers do wygenerowania embeddingu:

```
from sentence_transformers import SentenceTransformer

#####
# w przypadku problemów ze zmieszczeniem modelu na gpu #
#####
# import torch
# torch.cuda.is_available = lambda : False
# device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# print(device)
#####

model = SentenceTransformer("intfloat/multilingual-e5-large")

embeddings = model.encode([
    "To jest przykładowy tekst po polsku."
], normalize_embeddings=True)
```

2. Regulamin studiów AGH, po przekonwertowaniu do tekstu, należy podzielić na fragmenty (tzw. chunking) dwiema metodami:

- a) po 1000 znaków z przesunięciem co 200 znaków
- b) ustępami paragrafów

3. Dla każdego fragmentu przygotować jego reprezentację numeryczną (pkt. 1) – w efekcie Regulamin studiów jest przekonwertowany na listę słowników zawierających dwa klucze: 'text' z tekstem fragmentu i 'embedding' z wektorową reprezentacją fragmentu.

4. Dla kilku przykładowych pytań (np. "Jak jest zorganizowany rok akademicki w AGH", "Jak wygląda egzamin dyplomowy w AGH") przygotować ich wektorowe reprezentacje. Wyszukać metodą brute force fragmenty Regulaminu, które są semantycznie najbardziej podobne do pytań (tzn. iloczyn skalarny reprezentacji pytania i

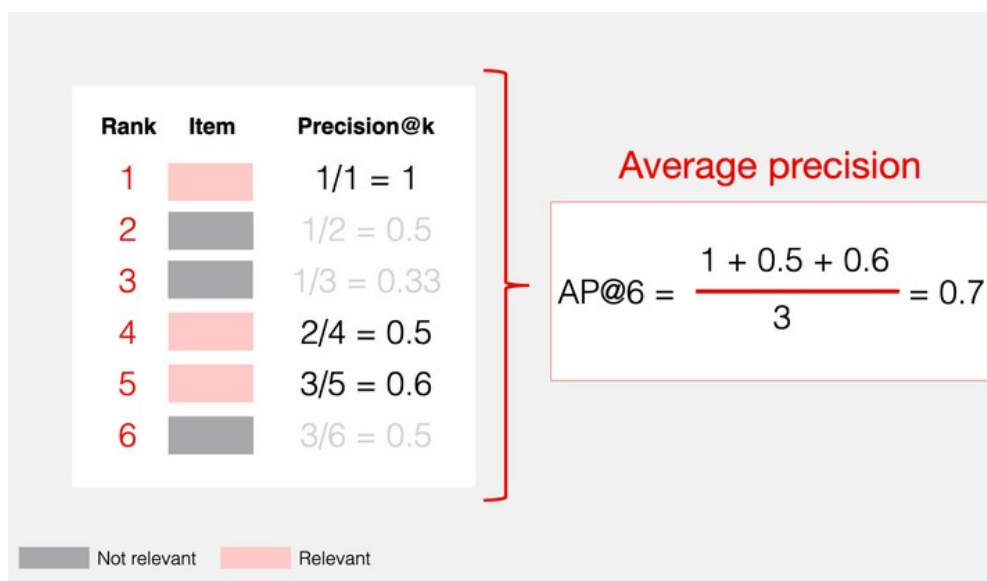
reprezentacji fragmentu jest jak największy). Dla 10 przykładowych zapytań ocenić średnią jakość wyszukiwania z wykorzystaniem metryk: precision@k i mean average precision@k, dla k=5.

Metryki precision@k i mean average precision@k są jednymi z metryk używanych do oceny skuteczności systemów klasy „Information Retrieval”. Wyznaczenie obu metryk wymaga sklasyfikowania przez osobę oceniającą wyszukanych dokumentów, jako relevant/irrelevant.

Sposób wyznaczenia metryki precision@k (<https://www.evidentlyai.com/ranking-metrics/precision-recall-at-k>, dostęp 22.05.2026):



Sposób wyznaczenia metryki mean average precision@k (MAP@k, <https://www.evidentlyai.com/ranking-metrics/mean-average-precision-map>, dostęp 22.05.2026):



$$\text{MAP@K} = \frac{1}{U} \sum_{u=1}^U \text{AP@K}_u$$

W powyższym wzorze indeksowanie w sumie przebiega po wynikach wyszukiwania.

5. Porównać jakość wyszukiwania dla dwóch rodzajów podziału regulaminu studiów na fragmenty.

Zadanie: Implementacja wyszukiwarki semantycznej „brute force” w oparciu o wektorowe reprezentacje tekstów. Ocena jakości wyszukiwania w oparciu o metryki „information retrieval” i porównanie metryk dla dwóch sposobów przygotowania zbioru tekstów.

Zadanie 4: Wyszukiwanie tekstowe, semantyczne i hybrydowe w langchain

Cel: Demonstracja metod wyszukiwania z wykorzystaniem biblioteki langchain.

Wyszukiwanie dokumentów jest kluczową funkcjonalnością wykorzystywaną przez systemy klasy RAG (Retrieval Augmented Generation) – informacja zawarta w wyszukanych dokumentach stanowi kontekst w oparciu o który model językowy przygotowuje odpowiedź na pytanie zadane przez użytkownika. Wysoka jakość wyszukiwania jest kluczowa dla jakości wygenerowanej odpowiedzi.

1. Studenci instalują pakiety wymagane do przeprowadzenia wyszukiwania:

```
pip install langchain_community
pip install rank_bm25
pip install faiss-cpu
```

2. Wyszukiwanie tekstowe w oparciu o podobieństwo leksykalne BM25
(https://pl.wikipedia.org/wiki/Okapi_BM25, dostęp 22.05.2026):

```
from langchain_community.retrievers import BM25Retriever
from langchain_core.documents import Document

# Dokumenty
docs = [
    Document(page_content="LangChain is a framework for LLM apps."),
    Document(page_content="BM25 is a lexical search algorithm."),
    Document(page_content="Chroma is a vector database."),
]

# Tworzenie BM25 retriever
retriever = BM25Retriever.from_documents(docs)

# Liczba wyników
retriever.k = 2

# Wyszukiwanie
query = "What is BM25?"
results = retriever.invoke(query)
```

```
# Wyniki
for doc in results:
    print(doc.page_content)
```

3. Wyszukiwanie semantyczne w oparciu o wektorowe reprezentacje tekstów:

```
from sentence_transformers import SentenceTransformer
from langchain_community.vectorstores import FAISS
from langchain_core.documents import Document
from langchain_core.embeddings import Embeddings

# -----
# Wrapper na SentenceTransformer
# -----

class SentenceTransformerEmbeddings(Embeddings):
    def __init__(self, model_name):
        self.model = SentenceTransformer(model_name)

    def embed_documents(self, texts):
        embeddings = self.model.encode(texts, normalize_embeddings=True)
        return embeddings.tolist()

    def embed_query(self, text):
        embedding = self.model.encode(text, normalize_embeddings=True)
        return embedding.tolist()

# -----
# Inicjalizacja modelu
# -----

embedding_function = SentenceTransformerEmbeddings(
    model_name="all-MiniLM-L6-v2"
)

# -----
# Dokumenty
# -----

texts = [
    "LangChain is a framework for LLM applications.",
    "FAISS is a vector similarity search library.",
    "Sentence Transformers create embeddings.",
    "Vector databases are used in RAG systems."
]

documents = [
    Document(page_content=t)
    for t in texts
]

# -----
# Utworzenie indeksu wektorowego FAISS
```

```

# -----

vector_store = FAISS.from_documents(
    documents=documents,
    embedding=embedding_function
)

print("FAISS vector store created.")

# -----
# Zapytanie do indeksu
# -----

query = "What creates embeddings?"

results = vector_store.similarity_search(
    query,
    k=2
)

for doc in results:
    print(doc.page_content)

# -----
# Wektorowy retriever
# -----

retriever = vector_store.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 3}
)

docs = retriever.invoke(
    "Explain vector search"
)

for d in docs:
    print(d.page_content)

```

4. Wyszukiwanie hybrydowe w oparciu o zespół retriever'ów (fuzja wyników wyszukiwań algorytmem Reciprocal rank Fusion):

```

from langchain_classic.retrievers import EnsembleRetriever

ensemble = EnsembleRetriever(
    retrievers=[bm25_retriever, vector_retriever],
    weights=[0.5, 0.5]
)

results = ensemble.invoke("What is LangChain?")

for doc in results:
    print(doc.page_content)

```


5. Dla pytań, jak w Zadaniu 3 wykonać wyszukiwania w zbiorze ustępów Regulaminu Studiów AGH, korzystając z metod w pkt. 2, 3 i 4. Ocenic wyniki wyszukiwania, korzystając z metryk wskazanych w Zadaniu 3.

Zadanie: Implementacja wyszukiwarki leksykalnej, semantycznej i hybrydowej z wykorzystaniem biblioteki langchain. Ocena jakości wyszukiwania w oparciu o metryki „information retrieval”.